Week-4 Assignment

1. SOLID PRINCIPLE

SOLID is a collection of five object-oriented design principles that help developers create software that is easy to understand, maintain, and extend. These principles improve code quality and reduce the chances of introducing bugs when applications grow in size.

The **Single Responsibility Principle (SRP)** states that a class should have only one responsibility and one reason to change.
For example, a UserService class should handle user-related operations, while an EmailService class should handle sending emails. If both functionalities are placed in the same class, maintaining the code becomes difficult.

The **Open/Closed Principle (OCP)** states that software entities should be open for extension but closed for modification. This means new features should be added without changing existing code.
For example, in a payment application, a new payment method such as UPI can be added by creating a new payment class instead of modifying the existing payment processing code.

The **Liskov Substitution Principle (LSP)** states that a subclass should be able to replace its parent class without affecting the correctness of the program.
For example, if a program expects a bird that can fly, replacing it with a penguin would violate this principle because penguins cannot fly.

The **Interface Segregation Principle (ISP)** states that a class should not be forced to implement methods it does not use.
For example, a robot should not be required to implement an eat() method if it only performs work-related tasks.

The **Dependency Inversion Principle (DIP)** states that high-level modules should depend on abstractions rather than concrete implementations.
For example, a user management system should interact with a database interface rather than directly depending on a specific database such as MySQL. This makes it easier to switch databases in the future.

2. Authentication Types

Authentication is the process of verifying the identity of a user before granting access to an application or system. Various authentication mechanisms are used in modern software applications to ensure security and protect user data.

Session-Based Authentication

Session-based authentication is one of the traditional methods used in web applications. When a user logs in successfully, the server creates a session and stores the session information. A session ID is sent to the user's browser as a cookie. For every subsequent request, the browser sends this session ID back to the server for verification. For example, many banking websites use session-based authentication to maintain user login status during a browsing session.

JSON Web Token (JWT)

JSON Web Token (JWT) is a token-based authentication mechanism commonly used in modern web and mobile applications. After a successful login, the server generates a digitally signed token containing user information and permissions. The client stores this token and sends it with every request. The server verifies the token before granting access. For example, a React application communicating with a Spring Boot API may use JWT to authenticate users without maintaining server-side sessions.

OAuth 2.0

OAuth 2.0 is an authorization framework that allows users to grant limited access to their resources without sharing their passwords. For example, when a user clicks "Sign in with" Google on a website, the website redirects the user to Google's login page. After successful authentication, Google provides an access token that allows the website to access specific user information without exposing the user's password.

Single Sign-On (SSO)

Single Sign-On (SSO) allows users to log in once and access multiple applications without entering credentials repeatedly. For example, after signing into a Google account, a user can access services such as Gmail, Google Drive, and Google Meet without logging in separately to each service. This improves user convenience and reduces password management challenges.